

Applicant Manager – Project Documentation

Version 1.0 – Last Updated: August 16, 2025 Author: \Nikhil Singhal (Developer)

1. Executive Summary

Applicant Manager is an in-house **Applicant Tracking System (ATS)** and **Resume Database** designed and developed during my internship to streamline recruitment operations.

The platform enables HR and recruitment teams to:

- **Post job vacancies** with detailed requirements and custom hiring workflows
- **Collect rich applicant profiles** with skill-specific years of experience
- **Track applications** through multiple stages, from `applied` to `hired`
- **Maintain a historical hire log** for compliance, analytics, and reporting

The system is implemented as a modular **Streamlit** application with a **Firestore Realtime Database** backend and **Google Drive integration** for resume management. Authentication is handled locally with `bcrypt` password hashing and encrypted session cookies.

2. System Architecture Overview

Frontend / Application Layer

- **Streamlit** for rapid UI prototyping and deployment
- Modular multi-page structure for clear separation of features
- Dynamic, reactive widgets for skill selection, education hierarchy, and data filtering

Backend / Database

- **Firestore Realtime Database** stores applicants, jobs, applications, and hired records
- Real-time data syncing ensures instant UI updates after operations
- Indexed queries for filtering and performance

File Storage

- **Google Drive API** integration for storing resumes
- Local staging before upload
- Shareable URLs stored in applicant records

Authentication & Session

- `bcrypt`-hashed credentials in a local JSON store (`users.json`)
- Persistent sessions using `EncryptedCookieManager` with AES encryption

Utility Modules

- `firebase_helper.py` : Centralized CRUD operations
 - `upload_resume.py` : Google Drive upload and OAuth handling
 - `auth.py` : Authentication and session handling
-

3. Key Functional Modules

Dashboard

- KPI Cards: **Clients, Open Vacancies, Applicants, Job Posts**
- Pie Charts: Vacancies by Department & Work Mode
- Live data from Firebase for real-time stats

Add Applicant Form

- Dynamic **course** → **specialization** selection (with “Other” option)
- Skills entry with per-skill **Years of Experience (YOE)**
- Contact details, work preferences, current & expected CTC
- Resume upload to Google Drive

Add Job Form

- Job details, required skills, client, budget, vacancies
- Custom hiring process stages (anchored with “applied” and “hired”)
- Updates `/skills` and `/clients` nodes if new entries detected

Add Application

- Link applicant to a job manually or in bulk
- Records initial status `applied` and timestamp in `/activity`

Add Hired

- Accessible only for applications in `offered` stage
 - Pre-fills `job_id`, `applicant_id`, `application_id` (non-editable)
 - Captures offered CTC, joining date, notice period, and hire status
 - Stored in `/hired` node for permanent reference
-

4. Data Views

View Applicants

- Full table with search (`id`, `name`, `email`), sorting, and advanced filters (`skills`, `location`, `experience`)
- Clickable Applicant ID → Applicant Detail Page

View Jobs

- Card and table views
- Filters: status, location, department, date range
- Clickable Job ID → Job Detail Page

View Hired

- Table of all hired records with links to job and applicant details
 - Useful for audit and placement tracking
-

5. Detail Pages

Applicant Detail

- **Tab 1 – Details:** Full profile + resume link, delete option
- **Tab 2 – Update:** Edit details and upload new resume

- **Tab 3 – Applications:**
 - Card layout with hiring stage stepper
 - Actions: Advance, Reject/Revert, Add Hired, Delete Application

Job Detail

- **Tab 1 – Description:** Job overview and delete option
- **Tab 2 – Applications:** Filterable application table, bulk advance
- **Tab 3 – Candidate Pool:** Applicants not applied to this job, with filters and bulk apply
- **Tab 4 – Update:** Edit job post

6. Data Model & Dictionary (Firebase)

/applicants

Field	Type	Description
id	string	Unique numeric ID
name, phone, email	string	Contact info
course, specialization, institute	string	Education info
skills	map	{ skill: YOE }
current_mode, current_duration	string	Current work status
preferred_mode, preferred_duration	string	Preferred work style
current_ctc, expected_ctc	float	Salary details
experience	float	Total YOE
resume_url	string	Google Drive link
created_at, updated_at	timestamp	Metadata

/jobs

Field	Type	Description
id	string	Job ID
job_title, department, location	string	Job details
skills	list	Required skills
hiring_process	list	Ordered list of stages
vacancies	int	Openings count
budget	string	Salary range/budget
client	string	Client name
status	string	Job status

/applications

Field	Type	Description
id	string	Application ID
job_id, applicant_id	string	Foreign keys
applied_at	timestamp	Application date
status	string	Current hiring stage
rejected	string	"true" / "false"
activity	map	Stage → timestamp

/hired

Field	Type	Description
id	string	Hire ID
job_id, applicant_id, application_id	string	References
hired_on, joining_date	timestamp	Hire dates
offered_ctc	float	Salary offered
status	string	Joined/Declined/Other
comments	string	Notes

7. Core Processes

1. **Job Creation** → /jobs record created, updates /skills and /clients
2. **Applicant Creation** → Resume uploaded to Drive, /applicants record created
3. **Application Linking** → /applications record created with applied status
4. **Hiring Stage Management** → Status updated + activity timestamp logged
5. **Hired Record Creation** → Minimal /hired entry stored for permanent record

8. Technical Highlights & Achievements

- **100% Dynamic Input Controls:** Live-updating skill & education fields without page reloads
- **Database Normalization:** Reduced redundancy by linking job_id & applicant_id instead of duplicating data in /hired
- **Resume Upload Automation:** Integrated Google Drive API to handle secure file uploads & retrieval
- **Configurable Hiring Stages:** Enforced consistent workflow while allowing customization
- **Real-time Data Sync:** Firebase listeners keep UI in sync without manual refresh

9. Security & Compliance

- Passwords stored as bcrypt hashes; never in plaintext

- Cookies encrypted with `COOKIES_PASSWORD` from environment vars
 - Firebase credentials stored securely and never committed to source control
 - Resume links protected by controlled Google Drive folder permissions
-

10. Future Improvements

- Migrate auth to **Firebase Authentication** with RBAC
 - Add **audit trails** for user actions
 - Enable **CSV/Excel export** for reporting
 - Integrate **resume parsing** (NLP) for auto-filling fields
 - Add **server-side pagination** for large datasets
-

11. Deployment & Setup

1. Install Dependencies:

```
pip install -r requirements.txt
```

2. Set Environment Variables in `.env` :

```
cred_file=/path/to/firebase-service-account.json
folder_id=GOOGLE_DRIVE_FOLDER_ID
user_file=users.json
COOKIES_PASSWORD=STRONG_SECRET
```

3. Authenticate Google Drive:

```
python utils/upload_resume.py
```

4. Run App:

```
streamlit run main.py
```